PA#1 - The ChatGPT Scheduler

3/13/2026

100 Points Possible

Attempt 1

In Progress
NEXT UP: Submit Assignment

Add Comment

Unlimited Attempts Allowed
3/15/2026

Details

Introduction

You are tasked with implementing three process scheduling algorithms: FIFO (First In, First Out), Pre-emptive SJF (Shortest Job First), and Round Robin in **Rust** (https://doc.rust-lang.org/book/title-page.html), but using ChatGPT. ChatGPT's implementation should be able to simulate the scheduling of multiple processes under each algorithm and calculate their turnaround time, response time, and wait time.

ChatGPT's implementation should include the following components:

1. A data structure to represent a process, including its arrival time, execution time, and status.
2. A scheduler function for each algorithm that takes in a list of processes and implements the chosen scheduling algorithm.
3. A time slice parameter (Q-value) for Round Robin, which determines how long each process should be allowed to run before being preempted.
4. A function to calculate our standard metrics: turnaround time, waiting time, and response time for each process.

You will be provided with test inputs and outputs to use as a benchmark for your results.

Each member of your team will be responsible for providing at least one prompt. You are allowed multiple iterations. You may continue to refine your prompt as much as necessary in order to get desirable results. You are encouraged to compare prompts as a team to arrive at a "best solution" for final submission. However, you will be required to keep and track your prompt history (see below).

At some point, it may seem like continuing prompt iteration yields diminishing returns. At your discretion, you may choose to stop prompting and complete the project on your own. Ideally, human-written code would be limited to output formatting and other superficial, minor tweaks. However, there may be some features or aspects of the project which perhaps ChatGPT refuses to generate. These should be clearly documented. All cases of human-generated code should be commented as such.

Finally, as a caution -- ChatGPT, at least as implemented at OpenAI.com, will occasionally refuse to generate code literally telling you that it can't. You can rephrase your prompt or even just ask it to carry it out regardless ("You've done this before. Please do it now").

Input File Format

Example:

```
processcount 3  # Read 3 processes
runfor 20       # Run for 20 time units
use sjf
process name A arrival 0 burst 5
process name B arrival 1 burst 4
process name C arrival 4 burst 2
end
```

directive	definition
scount	Number of processes in the list
runfor	How many time ticks to run in total
use	The algorithm to use. Valid values: fcfs, sjf, rr

directive	definition
quantum	(for rr only) length of the quantum in time ticks
process	see below
end	end of file marker

The process takes three named parameters:
name: The name of your process. This can be any valid string.
arrival: The arrival time of the process
burst: Total burst time

Remember to check the validity of the use and quantum parameters:

- If use specifies 'rr' and there is no quantum specified, print an error message and exit.
- In general, if any of the required parameters are missing, print an error and exit.
  - Error message should be in the form of: Error: Missing parameter <parameter>.
  - In the case of missing quantum parameter when use is 'rr' print: Error: Missing quantum parameter when use is 'rr'
- If the input file is not provided:
  - Print a usage message: Usage: scheduler-get <input file>

Output File Format

Example:

```
3 processes
Using preemptive Shortest Job First
Time   0 : A arrived
Time   0 : A selected (burst   5)
Time   1 : B arrived
Time   4 : C arrived
Time   5 : A finished
Time   5 : C selected (burst   2)
Time   7 : C finished
Time   7 : B selected (burst   4)
Time  11 : B finished
Time  11 : Idle
Time  12 : Idle
Time  13 : Idle
Time  14 : Idle
Time  15 : Idle
Time  16 : Idle
Time  17 : Idle
Time  18 : Idle
Time  19 : Idle
Finished at time  20

A wait   0 turnaround   5  response 0
B wait   6 turnaround  10  response 6
C wait   1 turnaround   3 response 1
```

Output must include all of the following:

- Number of processes
- The algorithm used
- When Round Robin is selected, print the Quantum on a separate line immediately following the display of the algorithm used.
- Every time tick which experiences an event (i.e. processes arrived, selected, and finished)
- When the CPU is not used, print "Idle"
- Note the last time tick.
- In your experimentation, you may have processes that don't complete within the given runtime. List those unfinished processes on the final summary line as

```
P1 did not finish
```

The example output shows all numerical outputs neatly right-justified. You do not need to do so. Whitespace between elements will suffice without fancy alignment. i.e.:

```
Time  1
Time  10
```

Handling input and output

Input

Your script must accept a single command-line parameter for the input filename. i.e.:
`scheduler-gpt inputFile.in`

Filename must be the only and *required* parameter. The input filename should have the extension of ".in".

Output

Your script will write out the output file as the base filename with the extension of ".out" i.e.:
`inputFile.in` should result in an output file called `inputFile.out`

Test Files

pa1-testfiles.zip (https://webcourses.ucf.edu/courses/1501090/files/119712977?wrap=1) (https://webcourses.ucf.edu/courses/1501090/files/119712977/download?download_frd=1)

Deliverables

- Each member of the team must provide link(s) to their conversation history. We want to see how the discussion evolves over time as you refine your query. Each individual conversation should be submitted as a link to chat.openai.com:
  - Each conversation is listed on the left-hand side. Each entry has a menu button. Select that, and then select "Share." You can then copy a link to the conversation.
- The team will then evaluate each conversation and determine the best path forward to the final code. This could be by picking the best result from the conversations submitted or it could be a hybrid of all of them. Whatever method is chosen, it should be documented and justified. Why and how did you arrive at the final result? This should be documented in your final report.
- The final report in which the team evaluates not only the final output, but the process of coding via generative AI. Only a single report is required for the entire team, in either Word or PDF format. All team members should be listed at the top followed by all links to the ChatGPT conversations. It should be titled "Group-N Final Report.docx" (or .pdf) where N is your Group Number. See below for the evaluation rubric.
- The final code to be evaluated as a single Rust source file named `scheduler-gpt.rs` (compiled to an executable on our end, of course). Team member names should be listed at the top of the file as comments.

Evaluation of AI Code

When evaluating AI-generated code, it's important to keep in mind that the code may not always be optimal or free of errors, and that human review is still necessary to ensure that the code meets the intended criteria and requirements. You will provide a written report of your team's evaluation of ChatGPT's output using the following rubric:

1. Correctness: Does the code perform the intended task correctly? Are there any bugs or errors that need to be fixed?
2. Efficiency: Is the code efficient and avoid unnecessary computations or data structures? Can the code be optimized for better performance?
3. Readability: Is the code well-organized, well-documented, and easy to understand? Does the code follow best practices such as using meaningful variable names, avoiding code duplication, and use a consistent coding style between prompts?

4. Completeness: Does the code handle edge cases and error conditions appropriately? Is the code flexible enough to handle different input data and scenarios? What happens when you don't feed it an input file or a malformed input file? Does the code account for race conditions (i.e. when two processes can technically be scheduled at the same time)?

5. Learning Assistance: Did you need AI assistance or additional support in learning the language? If so, please provide your prompts and a summary of the responses. Some of you may have had enough success using AI that maybe you felt you didn't need to learn Rust to get good results or perhaps the code was clear enough that you didn't feel the need to use additional resources to understand it. If this is the case, please describe. Do you feel that it's possible to use AI to code apps in languages you don't know or understand? What limitations do you foresee?

6. Overall Quality and Final Recommendation: Based on the above criteria, how would you rate the overall quality of the code? Would you recommend any changes or improvements to make the code more effective or efficient? How would you rate your overall experience writing code using the assistance of an AI? Was it easier or harder than you expected? What did you learn through the process? What would you do differently if you had to write code via AI again?

Grading Rubric

Correctness of Final Code (30 points)

- In order to avoid low-effort prompts and to encourage good code generation, your final output will be run against sample files and tested for their correctness. Your code will be run once against each of the algorithms. 10 points for each algorithm. Points will be deduced for incorrect outputs.

Prompt and Conversation Submissions from all Members (10 points)

- 4 points for all members contributing conversations, proportional points for partial submissions.
- 6 points for quality of conversations demonstrating an iterative process of refinement along with well-reasoned and well-documented explanations for how the final product came to be.

Addressing Each Point of Evaluation (60 points total)

- 10 points each. Points will be awarded on the basis of coherence and insight. Full points will demonstrate engagement with the topic and thoughtful, well-written response.

Bonus Points (10 points)

- Generated Code implements novel features such as:
  - Enhanced output rendering. This can be through the use of HTML, Markdown, or terminal codes to implement tables, colored text, or even animation.
  - Other scheduling algorithms like Linux's CFS.
  - Enhanced interface like a GUI.
- User-generated code may also implement new features. As this assignment is in its beta test stage, I would eventually like to compare the efficiency of AI-generated base code with human-written enhancements versus trying to let the AI write all of the code. Which is easier and/or faster?

View Rubric

ChatGPT Process Scheduler

Criteria	Ratings		Pts
Scheduler runs 2-process load on FCFS	**3 pts** **Full Marks**	**0 pts** **No Marks**	/ 3 pts
Scheduler Runs 5-process load on FCFS	**3 pts** **Full Marks**	**0 pts** **No Marks**	/ 3 pts
S[illegible]r runs 10-process lo[illegible] RR	**3 pts** **Full Marks**	**0 pts** **No Marks**	/ 3 pts

ChatGPT Process Scheduler

Criteria	Ratings				Pts
Scheduler runs 2-process load on SFJ (preemptive)	**3 pts** **Full Marks**		**0 pts** **No Marks**		/ 3 pts
Scheduler Runs 5-process load on SFJ (preemptive)	**3 pts** **Full Marks**		**0 pts** **No Marks**		/ 3 pts
Scheduler runs 10-process load on SFJ (preemptive)	**3 pts** **Full Marks**		**0 pts** **No Marks**		/ 3 pts
Scheduler runs 2-process load on RR	**3 pts** **Full Marks**		**0 pts** **No Marks**		/ 3 pts
Scheduler Runs 5-process load on RR	**3 pts** **Full Marks**		**0 pts** **No Marks**		/ 3 pts
Scheduler runs 10-process load on RR	**3 pts** **Full Marks**		**0 pts** **No Marks**		/ 3 pts
FCFS completes flawlessly	**1 pts** **Full Marks**		**0 pts** **No Marks**		/ 1 pts
SJF completes flawlessly	**1 pts** **Full Marks**		**0 pts** **No Marks**		/ 1 pts
RR completes flawlessly	**1 pts** **Full Marks**		**0 pts** **No Marks**		/ 1 pts
All team members contributed at least one ChatGPT conversation view longer description	**4 pts** **Full Marks**		**0 pts** **No Marks**		/ 4 pts
ChatGPT Conversation Quality view longer description	**6 pts** **Full Marks - Evidence of iteration accompanied by discussion** Multiple prompts were attempted showing a clear progression of conversation with ChatGPT. Prompts are accompanied by a discussion explaining their reasoning as to why they chose the prompts and how they decided when to stop prompting.	**4 pts** **Adequate iteration, no discussion** The team supplied a conversation that does show an attempt at engaging with ChatGPT, but failed to provide any discussion describing their iterative process.	**2 pts** **Minimal iteration, no discussion** There's minimal evidence that the team engaged with ChatGPT in an iterative fashion. They may have provided one or two different prompts, but did not provide any explanatory text or discussion of their iteration process.	**0 pts** **Did not engage with ChatGPT at all**	/ 6 pts
Evaluating the AI - Correctness view longer description	**10 pts** **Full Marks - Answered the questions with a detailed description of AI output and their response.**	**7 pts** **Addressed the questions, but provided minimal description.** Answered both questions, but did not provide much detail	**5 pts** **Answered both questions, but provided no additional details** Answered both questions, but did not provide a description	**0 pts** **No Marks**	/ 10 pts

ChatGPT Process Scheduler

Criteria	Ratings				Pts
	Answered both questions and additionally provided a detailed description of how ChatGPT did or did not meet their objectives. Description should include specifics on what code needed to be fixed and how they addressed it (whether by fixing it themselves or adjusting their prompting).	as to how the team addressed the issues. Some description is present of the team response, but it's vague.	of how the team responded to the AI's failures.		
Evaluating the AI - Efficiency view longer description	**10 pts** **Full Marks** This is a tough one and for now, we'll use a binary evaluation. Full points if they addressed the issue, none otherwise.		**0 pts** **No Marks**		/ 10 pts
Evaluating the AI - Readability view longer description	**10 pts** **Full Marks - All points addressed with some discussion on their findings** All questions answered with good descriptions of the AI output, supported by meaningful discussion and/or interpretation of the AI output.	**7 pts** **Addressed most of the points** Most of the points were addressed with meaningful discussion. They answered the question and provided some kind of commentary. "The code was easy to follow because it used verbose comments." or "The variable names made sense and were easy to read because it used camelCase". Some points may have been missed or not addressed.	**5 pts** **Answered the questions, but did not provide additional description or discussion** Answered the questions, but did not elaborate or interpret further. i.e. "The code was easy to understand and follow. The code used meaningful variable names." etc.	**0 pts** **No Marks**	/ 10 pts
Evaluating the AI - Innovation view longer description	**10 pts** **Full Marks** This is another binary evaluation as the question didn't prompt for further elaboration.		**0 pts** **No Marks**		/ 10 pts
Evaluating the AI - Final Evaluation view longer description	**10 pts** **Full Marks - All points addressed with some discussion on their findings** All questions answered with good	**7 pts** **Addressed most of the points** Most of the points were addressed with meaningful discussion. They	**5 pts** **Answered the questions, but did not provide additional description or discussion**	**0 pts** **No Marks**	/ 10 pts

ChatGPT Process Scheduler

Criteria	Ratings				Pts
	descriptions of the AI output, supported by meaningful discussion and/or interpretation of the AI output.	answered the question and provided some kind of commentary. "The code was easy to follow because it used verbose comments." or "The variable names made sense and were easy to read because it used camelCase". Some points may have been missed or not addressed.	Answered the questions, but did not elaborate or interpret further. i.e. "The code was easy to understand and follow. The code used meaningful variable names." etc.		
Evaluating the AI - Completeness view longer description	**10 pts** **Full Marks - All points addressed with some discussion on their findings** All questions answered with good descriptions of the AI output, supported by meaningful discussion and/or interpretation of the AI output.	**7 pts** **Addressed most of the points** Most of the points were addressed with meaningful discussion. They answered the question and provided some kind of commentary. "The code didn't initially account for error conditions, but after prompting it added them." or "The code handled race conditions well by adding checks for when two processes might conflict". Some points may have been missed or not addressed.	**5 pts** **Answered the questions, but did not provide additional description or discussion** Answered the questions, but did not elaborate or interpret further. i.e. "The code didn't handle any errors."	**0 pts** **No Marks**	/ 10 pts
					Total Points: 0

Keep in mind, this submission will count for everyone in your PA#1 Groups group.

Choose a submission type

Upload | Office 365 | More

Choose a file to upload

File permitted: DOCX, PDF, PY, HTML, TXT, MD, ZIP, RS

or

Canvas Files

(https://webcourses.ucf.edu/courses/1501090/modules/items/20515826)

Assignment

(https://webcourses.ucf.edu/courses/1501090/modules/items/205